

Author Harald Fjellström		Task VHDL2 Individual Task	
Email harald.fjellstrom@renewmic.com	Controlled	Version 1	File harald_fjellstrom_VHDL2_ingenjorsjobb_b.docx

AGSTU

Radix-2 FFT Core: Leveraging Modern Design Verification

AGSTU VHDL 2 - Individual Task

Harald Fjellström

2026-03-29

Summary: This project details the design and implementation of a flexible Radix-2 FFT core optimized for high mathematical accuracy and resource efficiency on FPGAs. By integrating a modern Python-based verification environment with automated testing, the design was rigorously validated against bit-accurate software models to ensure reliability. While the core is functionally verified, the synthesis results provide a clear roadmap for future hardware-specific mapping and timing optimizations to reach full operational speeds on physical silicon.

Content

1. Introduction.....	4
1.1 About the Fast Fourier Transform (FFT)	4
2. Requirement Specifications.....	5
2.1 Requirement Specification from Customer	5
2.2 Project-Specific Requirements	8
3. Earlier AGSTU Projects	9
3.1 FPGA-Based Speed Measurement with Incremental Pulse Generator	9
3.2 Radar Application Based on a LIDAR Sensor	9
4. Time Plan and Project Follow-up.....	10
4.1 Development Phases	10
4.2 Time Summary.....	10
5. Construction Description.....	11
5.1 System Architecture	11
5.2 Component Breakdown.....	11
5.2.1 Master Controller (fft_dif).....	11
5.2.2 The Complex Butterfly Unit (fft_butterfly).....	13
5.2.3 Memory and Utilities.....	13
5.3 Design Evolution & Rationale	14
5.3.1 Architectural Choices.....	14
5.3.2 Iterative Improvements Driven by Test Stimuli.....	14
6. Verification	16
6.1 Simulation Architecture and Data Flow	16
6.1.1 Data Movement and Synchronization.....	16
6.2 The Regression Suite: "Stress & Torture" Testing	16
6.3 Core Performance Metrics	17
6.4 Continuous Integration (CI/CD) with GitHub Actions.....	18
7. Synthesis and Validation (Hardware Focus).....	20
7.1 Timing Constraints (.sdc)	20
7.2 Synthesis Implementation.....	20
7.3 Synthesis Results and Bottlenecks	20
7.4 Synthesis Conclusion	21

8. Architectural Optimization Strategies	22
8.1 Common Platform Optimizations (Platform-Agnostic)	22
8.2 Altera MAX 10: High-Performance Parallelization	22
8.3 Lattice iCE40: Low-Power & Resource Efficiency	22
9. Appendices	24

1. Introduction

This project focuses on the implementation of a parameterizable Radix-2 Decimation-in-Frequency (DIF) Fast Fourier Transform (FFT) core using VHDL. The primary objective was to gain a fundamental introduction to the field of Digital Signal Processing (DSP) in hardware. The initial interest in this domain was sparked by reading *The Scientist and Engineer's Guide to Digital Signal Processing*. This text provided the necessary theoretical foundation for the project's hands-on engineering tasks. A link to the source can be found in **Appendix C**.

By utilizing the Python NumPy FFT implementation as a "golden reference," the project allowed for a robust learning experience through direct experimentation and bit-accurate comparison. As the project developed, the focus shifted from a purely hardware-centric implementation to a sophisticated simulation and verification framework. This shift was driven by the necessity for rigorous testing and the power of modern RTL verification tools, bridging the gap between high-level algorithmic study and low-level hardware design.

1.1 About the Fast Fourier Transform (FFT)

The Fast Fourier Transform is a highly efficient algorithm used to compute the Discrete Fourier Transform (DFT). Its primary function is to convert a signal from its original domain (usually time or space) into the frequency domain. By decomposing a signal into its constituent frequencies, the FFT reveals the spectral content of the data, which is often impossible to see in a raw time-domain waveform.

In modern engineering, the FFT is a cornerstone of Digital Signal Processing. It is used in a vast array of applications, including:

- **Telecommunications:** For modulation and demodulation in OFDM (WiFi and 4G/5G).
- **Audio Processing:** For equalization, compression (MP3), and pitch detection.
- **Radar and Lidar:** To calculate the velocity of objects via the Doppler shift.
- **Medical Imaging:** In MRI and CT scans to reconstruct images from raw sensor data.

Implementing the FFT in an FPGA (Field Programmable Gate Array) is particularly valuable for these applications because it allows for "Real-Time" processing—the ability to analyze the frequency spectrum of a signal as fast as it is being sampled, without the latency overhead of a general-purpose CPU.

2. Requirement Specifications

2.1 Requirement Specification from Customer

The following table outlines the foundational requirements provided for the assignment "AGSTU VHDL 2 Individual Task".

Table 1. Task Requirements

Krav id	Beskrivning	Utfört Ja/Nej
Förstudie/startrapport (lägg även med detta i slutrapporten)		
0	Beskriv två VHDL-projekt från tidigare år som helst har någon likhet med det som ska utföras i detta ingenjörsjobb.	Ja
1a	Aktivitets- och tidplan för projektet. Följ upp tidplanen och redovisa resultatet i slutrapporten. Lön är 500 SEK/tim.	Ja
1b	En separat kravspecifikationstabell. Beskriv det som ska realiseras med en inledning och en kravspecifikation. Kravspecifikationen får förändras under projektets gång.	Ja
1c	Gör en första arkitektur som visar alla ingående komponenter (kan senare modifieras). Beskriv också de komponenter som ska utvecklas, återanvändas från kursen och andras komponenter med referenser.	Ja
Leveranskrav av förstudie		
2	Startrapporten ska innehålla: Inledning Krav 0 (eget kapitel) Krav 1a (eget kapitel) Krav 1b (eget kapitel) Krav 1c (eget kapitel) Startrapporten ska skickas till chefen före projektstart för godkännande. Chefens måste budgetera pengar för projektet. Materialet återanvänds i slutleveransen. Leveransen ska ske till plattformen Itslearning. Namnet på filen ska vara "fornamn_efternamn_VHDL2_ingenjorsjobb_a.zip". Word-format krävs.	Ja
Förstudien avslutad -----		
Start av slutleverans		
Konstruktions krav		
3a	Efter kompilering av projektet, ska det inte finnas några "critical warnings" kvar. Om det finns varningar kvar ska dessa beskrivas.	Ja
3b	Konstruktionen skall vara robust, tex ta hand om risken för metastabilitet. Kod ska även vara lättläst, dvs indenterad i enlighet med det som lärts ut i vhdl-kurserna. Använd tex 3 mellanslag (2-4 mellanslag är rekommendationen). Inga TAB-tecken får förekomma i koden. Exempel på bra indenterad kod hittas i alla exempel i kursboken.	Ja
3c	Alla IP komponenterna ska vara i separata VHD-filer. Enbart en top fil.	Ja
3d	VGA-komponenten ska vara ingenjörens egenutvecklade.	N/A
Rapportkrav		
4	Utgå gärna från teknisk_rapport_mall, som ligger på Itslearning under kursen Konstruktionsmetodik och teknisk dokumentation. Kunden kräver en standardiserad teknisk rapport med filnamn	Ja

	"fornamn_efternamn_VHDL2_ingenjorsjobb". Följande ska finnas med i slutrapporten som ska vara i Word-format: • Titelsida med namn, datum och en sammanfattning. • Kravspecifikation, eget kapitel. Återanvänd från förstudier/startrapport med modifieringar. • Beskrivning av två äldre projekt (som liknar det valda projektet) med några kommentarer om förbättringar för att förstå konstruktionen, eget kapitel (krav 0). Återanvänd från startrapport. • Projektplan, aktiviteter och en budgeterad kostnad (500 SEK/tim), eget kapitel (krav 1a). Återanvänd från startrapport men lägg till uppföljning. • Testprotokoll för verifiering/validering, grundat på kravspecifikation 2 vilken togs fram av ingenjören i början. Protokollet behöver inte vara komplett, det räcker att visa hur det ska utföras och det behöver bara visas på systemnivå (inte på subnivåer), eget kapitel. • Konstruktions beskrivning, eget kapitel ○ Systemarkitektur ("top-down" beskrivning) ○ Delsystemsbeskrivning i underkapitel ○ I/O pinnar, Metastabilitetsproblem och sdc (gör en enkel sdc fil) • Verktygs- "settings och assignments", eget kapitel ○ Dokumentera alla förändrade kompilersinställningar för "settings och assignments" • Verifiering, eget kapitel ○ Teststimuli (som bygger på testprotokollet för konstruktionen) ○ Figurer med beskrivning på några viktiga resultat från simuleringen med ModelSim • Analys, eget kapitel ○ Felmeddelanden "Warning" från kompileringen som finns kvar, bör kommenteras. "Critical Warning" ska inte finnas kvar efter kompileringen ○ Max frekvens och storlek från Quartus-rapporterna ▪ Beskrivning av storlek för hela konstruktionen och varje delsystem (icke optimerad konstruktion) ○ Effektförbrukningsuppskattning ○ Tidsvalidering, SDC resultat med slack • Optimering, eget kapitel ○ Optimera konstruktionen ur storlek, visa före och efter optimeringen ○ Beskriv hur du har optimerat konstruktionen med hjälp av verktygen och eventuellt förändring i VHDL-koden • Validering, eget kapitel ○ Visa med resultat från testprotokollet (validering) ○ Kombinerar med en film (max 10 minuter) som beskriver för en utomstående vad som genomförts (visa gärna en systemarkitektur) och demonstrera att det fungerar (en länk till YouTube). Träna gärna på presentationsteknik, tänk på att det är en gymnasieelev som tittar på filmen (kunden granskar inte detta moment, mer än ser att det fungerar som det är tänkt).	
--	---	--

	OBS; Om du inte vill att filmen ska visas på Facebook med flera, skriv det efter länken. • Bilaga: Lista och kort beskrivning på IP-komponenter som kan återanvändas från projektet/kursen ○ IP komponenter som du har återanvänt och/eller skapat ska finnas i bilagan med följande struktur: Ip_komponent_namn ▪ Doc (om det finns dokumentation) ▪ HDL (VHDL filen) ▪ Test_bench (om det finns) ○ Motiveringen är att ska vara enkelt att återanvända dessa i nästa projekt	
Leveranskrav av slutleveransen		
5	Slutleveransen ska innehålla följande mappar med respektive dokument: • <u>Mappen "Dokumentation"</u> ○ Konstruktionsrapport i Word-format (se rapportkrav) ○ Eventuellt bilagor som kod, kretsschema och datablad • <u>Mappen "Design" (Konstruktionsunderlag för prototyp)</u> ○ Arkiverat Quartus projekt, ○ IP_komponenter ▪ Större komponenter, t.ex. VGA komponenten med rapporten (egenutvecklad) ▪ Andras IP komponenter som du använder med dokumentation och testbänk (om det finns) ○ Test bänkar • <u>Mappen "Konfigureringsfil"</u> ○ configuration.sof • <u>Mappen "Diverse"</u> ○ Datablad mm Leveransen ska ske till plattformen Itslearning. Den slutgiltiga leveransen är hela jobbet och namnet på filen ska vara "fornamn_efternamn_VHDL2_ingenjorsjobb_b.zip".	Ja*
6	Leveransen bedöms först ut teknisk synvinkel och komplettering kan göras maximalt 5 gånger om inte kunden (läraren) anser något annat. Därefter är det granskning av den tekniska rapporten och även här gäller det maximalt 5 kompletteringar för att få godkänt om inte kunden (rapportgranskaren) anser något annat. Om antalet kompletteringar överstiger dessa antal får den studerande ett helt nytt jobb att leverera till kund om läraren anser det motiverat.	

*Project structure is modified to suit the verification framework, see **Appendix A** for brief explanation.

2.2 Project-Specific Requirements

These requirements were established during the prestudy and updated to reflect the project's focus on simulation-driven verification.

Table 2. Project Requirements

Requirement ID	Description	Completed (Yes/No)
Design		
1	The FFT algorithm must be correctly implemented in hardware.	Yes
2	The FFT core must be dynamically parameterizable to support variable transform lengths	Yes
Verification		
3	FFT output (64 complex values) must be verified against reference Python implementation.	Yes
6	Verification must be performed using multiple input signals (e.g., harmonics, noise).	Yes
Synthesis		
4	The design must be fully synthesizable in Quartus for a MAX10 FPGA.	Yes
7	An analysis of potential efficiency improvements (e.g., Fmax or parallelization) shall be performed.	Yes
Documentation		
5	Any errors or deviations in calculation results must be analyzed and explained.	Yes

3. Earlier AGSTU Projects

3.1 FPGA-Based Speed Measurement with Incremental Pulse Generator

Author: Albin Hjälmsås (2020-03-15)

This project shares several common traits with the current work, particularly the implementation of a well-established and proven algorithm in hardware. The project is characterized by a clear link between theory and practical realization, where functionality is divided into well-defined hardware components. The associated report serves as a relevant reference for documenting a technical project, specifically regarding clarity and the structured presentation of theoretical backgrounds and validation methods.

3.2 Radar Application Based on a LIDAR Sensor

Author: Niklas Lind (2022-02-17)

While the direct technical connection between this project and the FFT implementation is limited, it was included as a reference because it handles sensor data and illustrates the application of technical concepts within sensor-related systems. The project highlights a practical approach driven by personal interest in a technical field. Although the documentation is less detailed in some respects, it demonstrates high ambition in both scope and results.

4. Time Plan and Project Follow-up

This chapter details the project activities and the distribution of effort across the development cycle. It should be noted that the development was carried out as evening work alongside a full-time internship. Consequently, while the logged hours are a best-effort representation of the work performed, the exact distribution of time contains a degree of uncertainty due to the fragmented nature of the schedule.

4.1 Development Phases

Algorithm and Implementation Study: Initial work involved studying signal representation in time and frequency domains, alongside a pre-study to define hardware requirements and core architecture choices.

- **First Iteration (19 hours):** Focused on achieving a mathematically correct baseline. While the initial VHDL was unoptimized, it established a functional state machine and arithmetic logic verified against software references.
- **VHDL Optimization and Feature Expansion (≈20 hours):** This phase refined the core for professional use, implementing variable sample sizes, IFFT support, and Block Floating Point (BFP) logic. A significant portion of this effort was dedicated to developing modular Python-based testbenches using the Cocotb framework. These testbenches were essential for isolating and debugging the functional integrity of the complete design. By verifying the RTL against NumPy-generated data within the simulation environment, these scripts ensured that existing logic remained stable as complexity increased and ultimately formed the foundation for the automated regression suite.
- **Jupyter Integration (≈5 hours):** Development of a Python-based analysis suite using the Cocotb testbench as a verification backend. This created an interactive "playground" for exploratory RTL simulation and visual data analysis. This environment allowed for the immediate comparison of VHDL wave-results against high-level mathematical models, facilitating rapid debugging of the DSP pipeline without leaving the Python ecosystem.
- **Verification and CI/CD Automation (≈8 hours):** Implementation of the full automated regression suite and a GitHub Actions pipeline. This system automates the execution of all test scenarios and generates timestamped PDF verification reports on every push to ensure ongoing mathematical integrity.
- **Final Documentation (≈10 hours):** Technical report writing together with synthesis and timing analysis.

4.2 Time Summary

The project represents a total effort of about **62 hours**. The transition from a "working baseline" to a parameterizable, automated verification framework ensures the core is robust across various FFT lengths and input scenarios. From a lifecycle perspective, the architecture is now ready for technology mapping; the clear separation between the top-level memory management and the encapsulated FSM/Arithmetic logic allows for the efficient replacement of generic VHDL components with vendor-specific primitives... without requiring a redesign of the core control flow.

5. Construction Description

The FFT core is built upon a modular, hierarchical architecture designed with a "simulation-first" methodology. This approach prioritized rigorous mathematical verification and algorithmic correctness in a high-level environment before transitioning to hardware-specific synthesis and timing optimizations.

5.1 System Architecture

The design utilizes a top-down structure centered on **encapsulation** and the clear separation of concerns. The `fft_top` entity serves as the primary structural wrapper, managing external interfaces and the dual-bank memory architecture.

Crucially, the `fft_dif` (Master Controller) **encapsulates** the `fft_butterfly` unit as a local sub-component. This nesting ensures that all complex arithmetic handshakes, twiddle-factor rotations, and timing-critical control signals remain local to the FSM. By hiding these low-level interactions, the core presents a simplified, abstracted interface to the rest of the system.

Component Hierarchy:

- **fft_top (Integration Layer):** The user-facing interface. It handles sample loading, execution triggering, and output monitoring, effectively abstracting the internal memory management and state transitions from the external system.
- **fft_dif (Master Controller):** The "brain" of the core. It implements the 8-state FSM that manages Decimation-in-Frequency (DIF) sequencing, complex memory stride generation, and Block Floating Point (BFP) scaling logic.
- **fft_butterfly (Arithmetic Sub-component):** Instantiated directly within the FSM, this is the core's processing engine. It is responsible for executing the complex additions, subtractions, and multiplications required for each butterfly operation.
- **fft_pingpong_ram (Memory Structure):** A unified memory block logically partitioned into dual banks. This architecture allows the FSM to read from one bank and write to the other simultaneously during each FFT stage, maximizing throughput and ensuring data integrity across the transform.

5.2 Component Breakdown

5.2.1 Master Controller (fft_dif)

The execution of the FFT is governed by an 8-state FSM that manages both the complex data pipeline and the intricate address generation required by the Radix-2 DIF algorithm.

To navigate the nested mathematical loops of the FFT while maintaining architectural clarity, the FSM relies on two primary hardware counters:

- **stage:** Tracks the outer loop (e.g., 0 to 9 for $N=1024$).
- **butterfly:** Tracks the total number of operations within a given stage (0 to $N/2 - 1$).

By utilizing a high-level behavioral modeling mathematical approach—prioritizing algorithmic clarity over manual bit-level optimization—the core dynamically translates these counters into memory indices. While this approach facilitated rapid prototyping and verified the indexing logic's correctness, the subsequent synthesis results confirm that these high-level operators are a part of the bottleneck for timing closure.

- **Address Generation (S0):** Calculates the memory indices for the current operation using standard VHDL arithmetic. `stride` and `span` are computed via integer division by powers of two ($N / (2^{(stage+1)})$), while `group_i` and `offset` are resolved using division (/) and modulo (mod) against the dynamic stride. This combinatorial logic cleanly maps to the theoretical DSP formulas. Once calculated, the FSM issues the first memory address for the butterfly pair.
- **Data Capture Pipeline (S1–S3):** To accommodate the 1-cycle latency of the block RAM, the FSM uses a pipelined approach:
 - **S1:** Issues the second memory address for the pair.
 - **S2:** Latches the first complex sample into the butterfly input registers.
 - **S3:** Latches the second complex sample, completing the data set for the arithmetic unit.
(See Figure 1 for the timing waveform demonstrating this synchronization).
- **Arithmetic Execution (S4):** Asserts the `en` signal to the Butterfly unit and waits in this state until the `done` flag is received.
- **Write-back (S5–S7):** Retrieves the processed samples from the Butterfly unit and writes them back to the memory indices. During this phase, overflow detection logic monitors the results; if any bit growth threatens the dynamic range, a flag is set to trigger Block Floating Point (BFP) scaling in the subsequent stage.
- **Bit-Reversal:** In the final computation stage, the FSM modifies its write-address logic. By applying a bit-reversal permutation to the destination addresses, the core ensures the output spectrum is stored in natural frequency order, eliminating the need for an external reordering buffer.

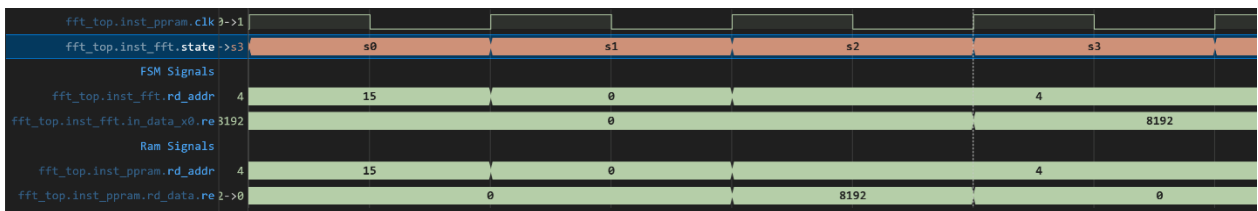


Figure 1. Data propagation delay to the FSM from the RAM

Ping-Pong Addressing Logic: To prevent data corruption during the read-modify-write cycle of each stage, the memory uses a Ping-Pong technique. A `bank_sel` register acts as the Most Significant Bit (MSB) for the memory addresses. As illustrated in the figure below, the FSM reads from Bank 0 and writes to Bank 1 during Stage 0, then toggles `bank_sel` to read from Bank 1 and write to Bank 0 in Stage 1, continuing this alternation until completion.

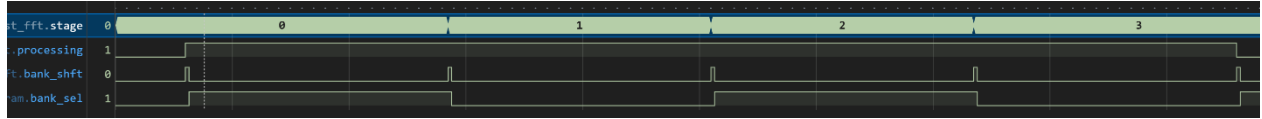


Figure 2. RAM bank selection toggle at the end of each stage

5.2.2 The Complex Butterfly Unit (fft_butterfly)

The Butterfly unit is the arithmetic heart of the core. Every operation is performed on complex numbers consisting of Real and Imaginary components. For a given pair of complex inputs X_0 and X_1 , and a complex twiddle factor W , the unit calculates:

$$Y_0 = X_0 + X_1$$

$$Y_1 = (X_0 - X_1) \cdot W$$

Internal Precision and Bit-Growth:

To maintain mathematical integrity, the unit does not operate strictly in Q1.14. Instead:

- **Resizing to Q2.14:** Upon entry, X_0 and X_1 are resized to 17 bits (Q2.14). This provides one extra integer bit of headroom, ensuring that the summation ($X_0 + X_1$) and subtraction ($X_0 - X_1$) cannot overflow.
- **Conditional Scaling:** If the FSM triggers the scale flag, the 17-bit result is right-shifted by 1 bit using convergent rounding to return the value to the standard 16-bit Q1.14 format.
- **Complex Multiplier:** The calculation of Y_1 utilizes four real-number multiplications and two additions.

Key Features:

- **Pipelining Ready:** The unit uses `en` and `done` handshaking, decoupling it from the FSM. This allows the master FSM to remain agnostic to the internal latency of the arithmetic unit, making future throughput optimizations trivial.
- **Fixed-Point Precision:** Implements rounded Q1.14 arithmetic with built-in support for the Inverse FFT (IFFT) by applying complex conjugate inversion to the twiddle factors when the `invert` flag is set.

5.2.3 Memory and Utilities

Ping-Pong RAM (fft_pingpong_ram): To align with the hardware constraints of the Lattice iCE40 UltraPlus—specifically its high-density SPRAM (Single-Port RAM) blocks—the memory is designed as a single contiguous array of `complex_q1_14` records. By using Single-Port architecture, the FSM is forced to sequence reads and writes across different clock cycles.

Twiddle Package: Contains the pre-calculated trigonometric factors generated via Python scripts. While currently implemented as a wide combinatorial lookup function for simulation flexibility, this has been identified as a primary bottleneck for hardware synthesis.

Utility Package: Defines custom VHDL types (like the `complex_q1_14` record) and provides mathematical helper functions like `log2_ceil` and `bit_reverse`, which are essential for making the FFT core dynamically parameterizable via VHDL generics.

5.3 Design Evolution & Rationale

The architecture of this FFT core was heavily shaped by specific hardware constraints and discoveries made during the iterative verification process.

5.3.1 Architectural Choices

Decimation-in-Frequency (DIF): The DIF algorithm was selected over Decimation-in-Time (DIT) primarily due to its input addressing. DIF accepts inputs in natural order and outputs in bit-reversed order, which streamlined the initial data-loading logic and FSM development.

Q1.14 Fixed-Point Format & Bit-Growth: A 16-bit word length was chosen to align with the 16x16-bit multipliers in the Lattice iCE40 DSP blocks. To prevent overflow, the Butterfly unit utilizes an internal Q2.14 precision (17 bits). Before any addition or subtraction, the X_0 and X_1 inputs are resized to include an extra integer bit.

This architecture allows the $X_0 \pm X_1$ operations to occur without clipping. If the scale flag from the previous stage's overflow detector is active, the unit applies Conditional Scaling (a 1-bit right shift) using convergent rounding to return the result to the standard Q1.14 format for storage. This ensures the design maximizes dynamic range while maintaining absolute mathematical stability across all stages.

Technical Note on Spectral Leakage: To maintain a strict focus on the core FFT butterfly and pipeline logic, pre-processing stages (such as Hann/Hamming windowing to mitigate spectral leakage) were intentionally excluded from this iteration. This ensures the verification environment tests the Radix-2 implementation in isolation, without external gain or phase artifacts.

5.3.2 Iterative Improvements Driven by Test Stimuli

During development, the core was subjected to a diverse battery of input signals designed to stress-test specific hardware edge cases. This "test-driven" approach ensured that every architectural feature was a direct response to a verified mathematical or physical requirement:

- **Dampened Sines (Full-Scale):** Testing with high-amplitude, dampened signals quickly identified overflow clipping within the Butterfly units during the initial stages. This discovery directly necessitated the implementation of the **Block Floating Point (BFP)** scaling logic to manage dynamic range across the entire transform.
- **Dirac Impulses and Pure Sinewaves:** While Dirac impulses were used to verify the basic scaling and impulse response, the simulation of Pure Sinewaves proved critical for frequency-domain validation. Since a pure sine should manifest as a single, discrete peak, any observed "spectral smearing" or bin-shifting immediately flagged synchronization errors in the master FSM. This led to a critical refinement of the 1-cycle latency model, ensuring that the data captured perfectly aligned with the complex addressing logic.

- **Low-Amplitude Signals:** Comparative analysis against the NumPy "Golden Reference" revealed a significant loss of precision (quantization floor) for small signals when fixed right-shifting was applied. This resulted in the development of Conditional Block Scaling, which monitors the headroom of the data set and only applies a shift when an overflow threshold is breached, preserving the Signal-to-Quantization-Noise Ratio (SQNR).
- **Large Sample Sizes (N):** As the FFT size increased toward N=2048, the accumulation of standard truncation errors became statistically significant. To mitigate this bias and improve the Effective Number of Bits (ENOB), Convergent Rounding ("round half to even") was implemented within the arithmetic unit, preventing a DC offset drift in the final output.

6. Verification

To ensure the mathematical integrity of the Radix-2 FFT core, a sophisticated, software-driven verification environment was developed. This allowed for bit-accurate comparison between the VHDL implementation and a "Golden Reference" (the NumPy FFT, specifically the 64-bit Double Precision Cooley-Tukey implementation) across a wide array of edge-case scenarios.

6.1 Simulation Architecture and Data Flow

The environment utilizes a multi-layered approach to bridge the gap between algorithmic theory and RTL execution:

- **Orchestration Layer (Python & Jupyter):** The top level manages test vector generation and final statistical analysis.
- **Middleware (Cocotb):** Acts as the co-simulation glue, allowing Python coroutines to drive VHDL signals, monitor the FSM state, and handle time advancement within the simulator.
- **Simulation Engine (NVC):** The backend compiler and execution engine that processes the VHDL source files.

6.1.1 Data Movement and Synchronization

Data is moved between environments using a structured file-passing mechanism.

- **Stimulus Phase:** A Python Jupyter notebook generates complex signals (e.g., Chirps or White Noise), scales them to Q1.14 fixed-point integers, and saves them as `.npz` binary files.
- **Execution Phase:** The Cocotb testbench loads these files, injects them into the `fft_top` entity, and triggers the FSM.
- **Reconstruction Phase:** Once the VHDL done signal is asserted, the hardware's internal RAM contents and the final Block Exponent are exported back to Python. The results are then "de-normalized" (shifted back by the exponent) to allow for a direct comparison with the floating-point reference.

```
# 2. Run Hardware FFT
# We pass the sampled array directly to run_sim function
fft_hw, exp1, cycles = run_sim(sig_sampled, invert=False)
# 3. Run Hardware IFFT
# We pass the FFT result back into the hardware to reconstruct the time domain
# Note: Since our run_sim handles quantization, we pass the complex fft_hw
sig_reconstructed, exp2, cycles_i = run_sim(fft_hw, invert=True)
# Note the scaling methodology to restore the absolute amplitude of the original
sig_reconstructed = sig_reconstructed * (2**(exp2-exp1)) / N
```

Figure 3. Python Orchestration: Initiating the hardware round-trip simulation via the Cocotb middleware

6.2 The Regression Suite: "Stress & Torture" Testing

Because different hardware bugs (such as bin offsets or scaling overflows) only appear under specific spectral conditions, a "battery" of scenarios was developed to stress the core:

- **Dirac Impulses:** Used to verify correct addressing and gain (bin offset detection).
- **Damped Sines:** High-amplitude signals used to stress the Block Floating Point (BFP) scaling logic.
- **Torture Tests:** "Goliath vs. David" tests, where a tiny signal is hidden next to a full-scale tone. This validates the core's ability to maintain a low quantization noise floor.
- **Pi/4 Stress:** Specifically targets the 45-degree twiddle multiplications to ensure rounding precision.

6.3 Core Performance Metrics

The suite automatically calculates the following metrics for every transform size from N=16 to N=2048:

- **SQNR (Signal-to-Quantization-Noise Ratio):** Measures the ratio between signal power and the noise introduced by rounding. Because the hardware uses a Q1.14 fixed-point format, the theoretical maximum SQNR is approximately $6.02 \cdot 14 + 1.76 \approx 86.04 \text{ dB}$. Therefore, a hard ceiling of 90.0 dB is used in the testbench to represent a mathematically "perfect" result without floating-point division-by-zero errors.

```
# 3. Calculate Mean Squared Error (MSE)
# MSE = average(|Golden - HW|^2)
error = golden_ref - hw_sig
mse = np.mean(np.abs(error)**2)

# 4. Calculate Signal-to-Quantization-Noise Ratio (SQNR)
signal_power = np.mean(np.abs(golden_ref)**2)

if mse == 0:
    # If the error is zero, the SQNR is effectively limited only by
    # the precision of the floating point numbers used for the check.
    # Theoretical max SQNR ~= 6.02 x 14 + 1.76 = 86.04 dB
    sqnr = 90.0 # Or any high number that represents "Perfect"
else:
    sqnr = 10 * np.log10(signal_power / mse)
    sqnr = min(sqnr, 90.0)
```

Figure 4. Python implementation of the SQNR calculation within the verification suite.

- **ENOB (Effective Number of Bits):** Translates SQNR into a bit-depth. A result of 14.66 bits is the targeted maximum for this 16-bit architecture (representing 90 dB "mathematical perfection" for Q1.14).
- **Block Exponent (Exp):** Tracks the number of arithmetic shifts performed. This validates that the BFP logic is actively preserving headroom and preventing overflow.
- **Cycle Count:** Measures the FSM efficiency. By tracking cycles across different N values, the core's computational complexity $O(n \log n)$ is verified in actual hardware cycles.

Scenario	Objective	Max SQNR	Avg ENOB	Max Exp	Status
Dirac Impulse	Tests basic timing and unity gain; no twiddle math involved.	90.00 dB	14.66	1	✓ PASS
DC & Nyquist	Checks sign-extension and accumulation at frequency extremes.	90.00 dB	14.66	11	✓ PASS
White Noise	Stresses the complex multipliers with random, high-energy data.	72.97 dB	11.07	8	✓ PASS
Linear Chirp	Tests phase-factor (twiddle) accuracy across all bins.	77.25 dB	11.88	7	✓ PASS
Pi/4 Stress	Validates 45-degree multiplications where Sine and Cosine are equal.	90.00 dB	14.66	11	✓ PASS
Damped Sine (Full)	Real-world transient; tests scaling logic with high dynamic range.	70.36 dB	10.39	8	✓ PASS
Damped Sine (-20dB)	Mid-scale signal; checks precision with 3-4 bits of head-room.	61.85 dB	9.16	5	✓ PASS
Damped Sine (-40dB)	Small-scale signal; identifies the quantization noise floor.	39.61 dB	5.91	1	⚠ WARN (Noise Floor)
Torture (-80dB)	Tests Processing Gain; detecting a tiny tone near a large one.	84.97 dB	13.10	11	✓ PASS
Torture (-95dB)	Extreme sensitivity test; signal buried below the LSB.	90.00 dB	13.58	11	✓ PASS
SYSTEM ROUND-TRIP	End-to-end FFT -> IFFT verification; validates unity gain and total error.	55.72 dB	8.96	N/A	✓ PASS

Figure 5. Automatically generated Verification Report summary table

6.4 Continuous Integration (CI/CD) with GitHub Actions

To ensure that no code change introduces regressions or mathematical errors, the verification suite is integrated into a GitHub Actions CI/CD pipeline.

- **Automated Triggering:** Every time code is "pushed" to the repository, a virtualized Linux environment is spun up.
- **Environment Setup:** The runner automatically pulls the repository and installs the NVC simulator, Python 3.10, and the necessary libraries to create a reproducible environment before executing the regression test suite.
- **Artifact Comparison:** A key feature of this pipeline is the ability to generate and store verification artifacts (reports and data) from different builds. This allows for a direct comparison of how specific VHDL changes affect the numerical precision of the core.
- **Precision Tracking Case Study:** An example of this utility can be found in the repository under *docs/reports/*, which contains a "Before and After" comparison of the Conditional First-Stage Scaling implementation. By comparing these artifacts, it was possible to visually and mathematically confirm that the change significantly improved the SQNR for low-amplitude signals without affecting the stability of full-scale inputs.

- **Mathematical Safeguard:** This process ensures that any optimization for speed or area (such as pipelining the butterfly) is verified against the 90 dB SQNR threshold before being merged into the main branch.

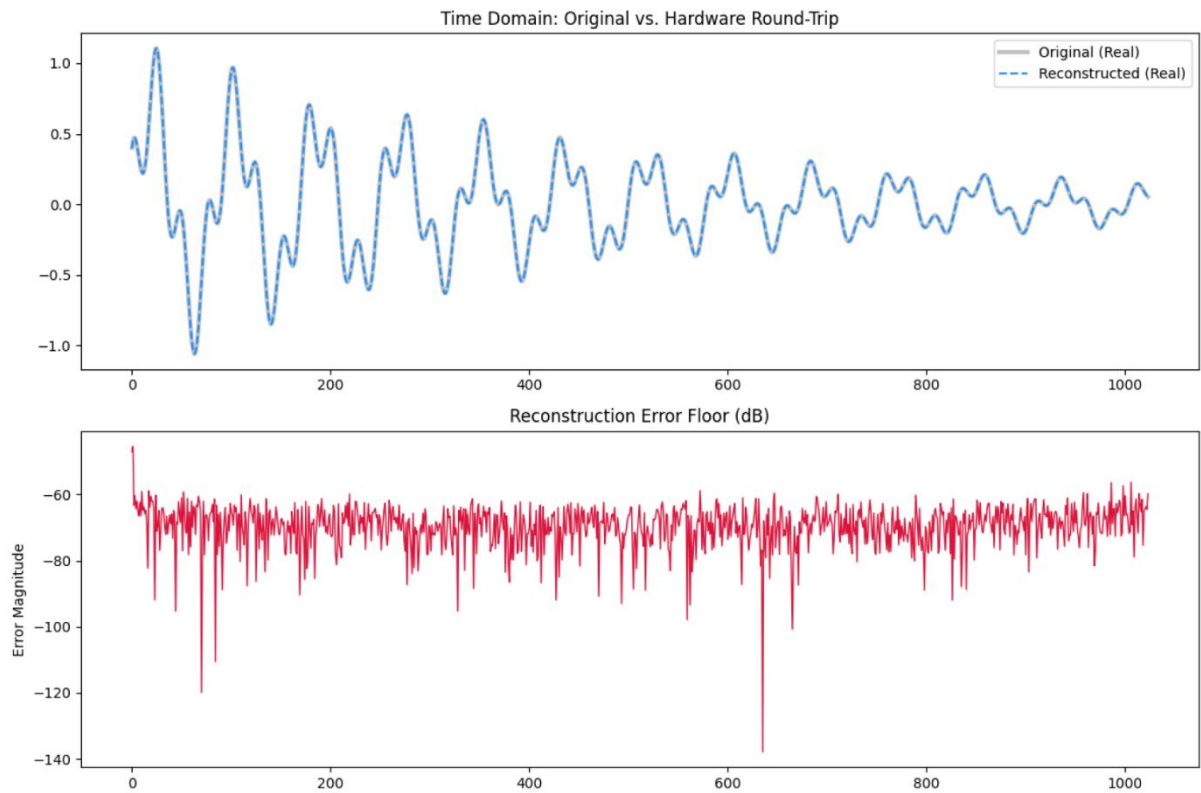


Figure 6. Round trip IFFT(FFT(X)) of 1024 sample signal.

7. Synthesis and Validation (Hardware Focus)

Synthesis tests were performed using Quartus 22.1 targeting the Altera MAX 10 (10M50DAF484C7G) FPGA found on the DE10-Lite development board. This process provided a critical "reality check" on how high-level VHDL translates into physical silicon.

7.1 Timing Constraints (.sdc)

To evaluate the design's performance, a minimal Synopsys Design Constraints (.sdc) file was created. By defining a 20.0 ns (50 MHz) period, the tool was able to identify exactly where the logic failed to meet the required setup times, providing the slack data used for the following analysis.

7.2 Synthesis Implementation

The design successfully synthesized with minimal adjustments. A key modification involved replacing certain VHDL-2008 conditional when statements inside clocked processes with standard if-else blocks. This change was necessary to comply with the Quartus synthesis engine's strict interpretation of synchronous logic templates.

7.3 Synthesis Results and Bottlenecks

The synthesis results highlighted several areas where the "simulation-first" code diverged from efficient hardware mapping:

- **The RAM Inference Issue:** Although the memory was architected to mimic the Single-Port RAM (SPRAM) found in iCE40 FPGAs, the Quartus engine did not automatically infer Altera M9K memory blocks. Instead, the 1024-word complex array was mapped to Registers (Logic Cells).
 - **Analysis:** This resulted in a massive resource footprint and high routing congestion. For the tool to recognize the RAM, the VHDL records likely need to be flattened into standard logic vectors, or the read/write logic must follow a strict template.
- **Timing and FMAX:** With the 50 MHz constraint applied, the timing analyzer reported a maximum frequency (FMAX) of approximately 7 MHz.
- **Critical Path (Twiddles):** The primary bottleneck is the Twiddle Lookup Table. The wide combinatorial "case" statement creates a logic depth that is too high for high-speed operation.

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-116.474	fft_dif.inst_fft[stage[0]	fft_dif.inst_fft[twiddle.im[13]	clk	clk	1.000	0.004	117.470

Figure 7. Twiddle Timing

- **Arithmetic Addressing Logic:** Consistent with the High-level Behavioral Modeling mathematical approach described in Section 5.2.1, the use of integer division (/) and modulo (mod) for address generation resulted in deep combinatorial paths, directly contributing to the 7 MHz FMAX bottleneck. While these operators evaluate perfectly in simulation, FPGA synthesizers can struggle to fully optimize variable-divisor operations into simple bit-shifts, resulting in deep, slow combinatorial logic paths between the FSM registers and the RAM address lines.

- **Butterfly Timing:** The arithmetic Butterfly unit also showed timing violations. Meeting the 50 MHz target will require internal pipelining (register stages) within the complex multiplier logic.

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	-14.963	fft_dif:inst_fft in_data_x0.im[0]	fft_dif:inst_fft_bf_inst y1.re[8]	clk	clk	1.000	-0.053	15.902

Figure 8. Butterfly Timing

7.4 Synthesis Conclusion

While the design is mathematically verified in simulation, these results provide a clear engineering roadmap for future hardware optimization. Transitioning to clocked ROM for twiddles, implementing arithmetic pipelining, and refining the RAM synchronous template are the required steps to reach targeted hardware speeds.

Flow Status	Successful - Mon Mar 23 17:28:29 2026
Quartus Prime Version	22.1std.2 Build 922 07/20/2023 SC Lite Edition
Revision Name	fft_top
Top-level Entity Name	fft_top
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	8,552 / 49,760 (17 %)
Total registers	4447
Total pins	85 / 360 (24 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	9 / 288 (3 %)

Figure 9. Synthesis report

8. Architectural Optimization Strategies

The bottlenecks identified during synthesis highlight that a "one-size-fits-all" HDL approach eventually hits physical limits. The following strategies represent the optimization paths the core can take depending on the target hardware's philosophy.

8.1 Common Platform Optimizations (Platform-Agnostic)

These improvements are required for any physical deployment to move beyond the current 7 MHz diagnostic baseline.

- **Synchronous Twiddle ROM:** Replace the wide combinatorial case statement with a clocked ROM. Transitioning to synchronous twiddle-factor fetches eliminates the significant logic depth that currently serves as the primary timing bottleneck. A further resource optimization would be the implementation of quarter-wave symmetry; by storing only 25% of the complex unit circle and using index reflection or sign-bit manipulation to derive the remaining values, the memory footprint can be reduced by 75% without sacrificing precision.
- **Hardware-Native Addressing Logic:** While modern synthesis tools can often optimize power-of-two division into shifts, explicitly implementing bit-slicing and concatenation removes any ambiguity in the logic synthesis. This ensures the shortest possible combinatorial path from the FSM state registers to the RAM address ports, minimizing 'routing jitter' and maximizing setup slack.
- **Strict RAM Inference Templates:** The current implementation uses VHDL records and custom logic that caused the synthesis engine to fall back to Logic Cell (register) mapping. To optimize resource usage, the memory entities must be rewritten to strictly match Vendor-Specific RAM Templates. This involves ensuring a purely synchronous read-path without asynchronous resets, which guarantees the tool correctly infers dedicated Block RAM (M9K for Altera, SPRAM for Lattice) rather than consuming thousands of registers.

8.2 Altera MAX 10: High-Performance Parallelization

For high-performance targets, the focus shifts toward maximizing throughput by leveraging abundant hardware resources.

- **Arithmetic Pipelining:** Insert register stages specifically between the pre-adder/subtractor and the complex multiplier, and again after the multiplier accumulation. This breaks the longest combinatorial chain, aligning perfectly with the internal pipeline registers of standard FPGA DSP blocks, allowing the arithmetic logic to achieve timing closure at 50–100 MHz.
- **Multi-Engine Parallelism:** Transition to True Dual-Port RAM and instantiate multiple parallel Butterfly units. This architecture allows the system to process several "wings" of the FFT stage simultaneously, effectively doubling or quadrupling bandwidth.

8.3 Lattice iCE40: Low-Power & Resource Efficiency

The strategy for the iCE40 platform is centered on strict resource budgeting and silicon efficiency.

- **SPRAM Sequencing:** Prioritize the use of high-density Single-Port RAM (SPRAM) to preserve limited Embedded Block RAM (EBR) for system-critical tasks like FIFOs. The current FSM's 1-cycle latency model is specifically designed to maintain pipeline activity while respecting SPRAM's sequential access nature.
- **DSP Slice Preservation:** Maintain the Q1.14 format to map perfectly into the 16x16-bit multipliers of the iCE40 DSP blocks. By sharing a single Butterfly unit across all stages, the design minimizes the MAC (Multiply-Accumulate) slice count, leaving hardware available for pre-FFT filtering or windowing.
- **Arithmetic Logic Simplification (Truncation):** The current implementation uses convergent rounding to prevent DC bias in the FFT results. For applications where resource utilization is more critical than absolute spectral purity (e.g., simple magnitude-based peak detection), this could be replaced with simple truncation. By simply dropping the fractional bits, the logic required for rounding increments and carry-chains is eliminated, significantly reducing the LUT count in the Butterfly unit at the cost of a slightly higher noise floor and a small DC offset.

9. Appendices

Appendix A (Project Repository): https://github.com/HaraldFjellstrom/VHDL_FFT

/docs: Contains this file and other documents

/notebooks: Notebooks and fft_utils.py helper library

/sim: Cocotb testbench

/vhdl: VHDL source code

/README.md: Project overview and how to set up the environment

Appendix B (Video Demonstration): TODO

Appendix C (Inspirational Source): Smith, Steven W. (1997). The Scientist and Engineer's Guide to Digital Signal Processing. (Available online at www.dspguide.com).